Git and CI/CD compact cheatsheet

STEP 1 Getting started with Git

1.1 What Git is and how to set it up

Version control Records every version of your files with author, time and message, so you can compare, restore and work in parallel.

Distributed Every Git clone has the full history. Commit, log and diff work offline. Centralised tools need the server.

Snapshots A commit is a snapshot of the whole project. Unchanged files are reused, not stored again.

Config levels system, global (~/.gitconfig), local (.git/config). Later wins. Set user.name, user.email and init.defaultBranch first.

Git is not GitHub Git is the tool. GitHub, GitLab and Bitbucket host repositories and add pull requests, reviews and CI/CD.

Set a value for your user
```
git config --global <key> "<value>"
```

Which value wins system < global < local

TRAP Running git init in the wrong folder, for example your whole home folder, makes Git track everything. If that happens, delete the new .git folder in that place. Your files are not touched.

EXAM Git and GitHub are not the same thing. Git is the tool on your computer. GitHub, GitLab and Bitbucket are websites that host Git repositories and add pull requests, issues and CI/CD pipelines.

1.2 The three areas: working tree, staging area and repository

Three areas Working tree (your files), staging area or index (next snapshot), repository (all commits). add moves changes to the index, commit saves the index.

Status codes git status -s prints XY: X staged (index vs HEAD), Y unstaged (folder vs index). ?? untracked, MM staged and changed again.

Commit saves the index Edits made after git add are not in the commit until you add again. commit -a stages tracked files only.

Partial staging git add -p stages hunk by hunk so one file can feed two clean commits.

Good messages Short imperative summary near 50 characters, blank line, then why. One logical change per commit.

Short status
```
git status -s → XY path
```

Stage
```
git add <file>, git add ., git add -p
```

Commit `git commit -m "message"`

TRAP git commit saves what is staged, not what is in your file. If you edit a file after git add, the newer edit is not in the commit until you add it again.

1.3 Reading history: log, diff, blame and naming commits

git log --oneline --graph --all for the picture. -S text to find when code appeared. -p and --stat for what changed. -- path for one file.

Three diffs git diff: folder vs index. git diff --staged: index vs HEAD. git diff HEAD: folder vs HEAD.

Hunks @@ -a,b +c,d @@: old lines a to a+b-1 became new lines c to c+d-1. A changed line is one - and one +.

Naming commits X~n: n first parents back. X^n: n-th parent of a merge. HEAD@{n}: where HEAD was n moves ago.

Ranges A..B: in B not in A. A...B: in one but not both (log), or merge base to B (diff).

Compact history
```
git log --oneline --graph --decorate --all
```

Find code
```
git log -S "price_limit"
```

Hunk header `@@ -a,b +c,d @@`

Ancestors
X~n = n first parents back. X^n = the n-th parent of X

Two dots
A..B = reachable from B but not from A

TRAP git diff with no arguments shows nothing after git add. The changes did not vanish: they moved to the index. Use git diff --staged.

EXAM HEAD~2 and HEAD^^ name the same commit. HEAD^2 is different: it is the second parent of HEAD and only exists if HEAD is a merge commit.

STEP 2 Branching and merging

2.1 Branches and HEAD

Branch A branch is a ref, a 41 byte file with a commit id. Creating one copies nothing. Commits move only the branch HEAD points to.

switch git switch name, git switch -c name, git switch - (previous). checkout does the same in older guides.

Detached HEAD HEAD points at a commit. Commits made there need git switch -c name, or they are left for the reflog only.

Delete git branch -d refuses unmerged work. -D forces. --merged and --no-merged help clean up.

Protect work Git refuses to switch if uncommitted edits would be overwritten. Commit or stash first.

Switch `git switch <name>`

Create and switch
```
git switch -c <name>
```

Delete safely `git branch -d <name>`

TRAP Commit or stash (chapter 3.2) before switching when Git complains. Never fix it with a hard reset unless you really want to lose the edits.

2.2 Merging and resolving conflicts

Fast-forward If the current branch has not moved, merge just moves the pointer. --no-ff forces a merge commit, --ff-only forbids one.

Three way merge Base, ours, theirs. One side changed: take it. Same change: take it. Different changes: conflict.

Resolve Edit, delete <<<<<<< ======= >>>>>>> markers, git add, git commit. git merge --abort to give up.

Strategies ort is the default since Git 2.34. -s ours keeps our tree, -X ours only settles conflicts our way.

Clean is not correct A merge without conflicts can still break the program. Test the merged result in CI.

Merge a branch into the current one
```
git merge <branch>
```

Merge base
```
git merge-base main feature
```

Abort `git merge --abort`

TRAP A file with conflict markers still compiles in some languages, for example inside a YAML string or a Markdown file. Search for <<<<<<< before committing, or let a pre-commit check do it.

EXAM A clean merge is not a correct merge. Two changes can merge without any conflict and still break the program together. That is why CI on pull requests tests the merged code, not only the branch.

2.3 Rebase, interactive rebase and cherry-pick

Rebase Replays your commits on a new base. Copies get new ids. --continue after fixing, --abort to give up.

Golden rule Never rebase commits others may have built on. Rebase your own branch, then push --force-with-lease.

Merge or rebase Merge keeps true history and is safe on shared branches. Rebase gives a straight line for your own work.

Interactive rebase rebase -i: pick, reword, edit, squash, fixup, drop, reorder. commit --fixup plus --autosquash automates it.

Cherry-pick Copies one commit as a new commit, for example a fix to a release branch. -x records the source.

Rebase the current branch
```
git rebase <upstream>
```

Continue, skip or stop
`git rebase --continue`, `--skip`, `--abort`

Prepare a fixup now, squash later
`git commit --fixup=<commit>` then `git rebase -i --autosquash <base>`

Copy commits
`git cherry-pick <commit>...`, `git cherry-pick A..B`

TRAP Rebased commits are copies with new ids, because their parent changed. Anyone who had the old ones now has commits that no longer exist on the branch.

TRAP After rebasing a branch you already pushed, a normal push is rejected. Use --force-with-lease, never plain --force, and only on a branch that is yours. Chapter 4.1 explains why.

STEP 3

Undo, stash, tags and ignore rules

3.1 Undoing things: restore, reset, revert and the reflog

Where is it Unstaged edit: git restore file. Staged: git restore --staged file. Local commit: reset or amend. Shared commit: revert.

Reset modes --soft moves the branch only. --mixed (default) also resets the index. --hard also resets your files.

Revert Makes a new inverse commit. Safe on shared branches. Merge commits need -m 1.

Reflog Local diary of HEAD. git reflog, then git reset --hard HEAD@{n} or git branch rescue . Unreachable entries expire after 30 days by default.

Danger reset --hard and restore throw away uncommitted edits, which no command can bring back.

Undo the last commit, keep the work staged

```
git reset --soft HEAD~1
```

Discard all local work

```
git reset --hard HEAD
```

Undo a pushed commit

```
git revert <commit>
```

TRAP git restore app.py without --staged overwrites your file from the index. Uncommitted edits it removes are not in any commit, so the reflog cannot bring them back.

TRAP git reset --hard with uncommitted changes is the most common way people lose work in Git. Run git status and git stash first if in doubt.

3.2 Stash, tags, .gitignore and .gitattributes

Stash git stash (-u for untracked), stash list, pop (apply and drop), apply (keep), stash branch. A WIP commit is often safer.

Tags Lightweight = plain ref, annotated (-a -m) = tag object for releases. Push with git push origin or --tags. Never move a shared tag.

Semver MAJOR.MINOR.PATCH: breaking, feature, fix. Lower parts reset to 0.

.gitignore Last match wins, ! re-includes, leading / anchors, trailing / means folder, no re-include inside an ignored folder. check-ignore -v explains.

Tracked stays tracked Ignore rules do not untrack files: git rm --cached. A committed secret is leaked: rotate it.

Save and clean

```
git stash or git stash push -m "note"
```

Bring back

```
git stash pop, git stash apply
```

Push tags

```
git push origin v1.0.0, git push --tags
```

Semantic versioning

MAJOR.MINOR.PATCH

Why is this file ignored?

```
git check-ignore -v <path>
```

Already committed by mistake

```
git rm --cached <file> then commit
```

TRAP Moving a tag that others have fetched is like changing a published book's edition number. Do not do it. Make a new tag instead.

TRAP If a secret was ever committed and pushed, removing it in a new commit is not enough: it is still in history. Treat it as leaked and rotate the password or key at once.

STEP 4 Remotes and teamwork

4.1 Remotes: clone, fetch, pull and push

Remote basics clone creates origin and origin/* remote-tracking branches. origin/main is your last known copy of the server branch, updated only by fetch, pull and push.

Fetch, pull, push fetch downloads and changes nothing else. pull = fetch + merge or rebase (pull.rebase, --ff-only). push is accepted only as a fast forward.

Ahead and behind ahead = commits only you have, behind = commits only the server has. Both above 0 means diverged.

Upstream git push -u origin sets tracking. git branch -vv shows it with ahead and behind counts.

Force safely Never force push shared branches. On your own branch use --force-with-lease (and --force-if-includes). fetch --prune removes stale origin/* names.

Clone `git clone <url> [folder]`

Set upstream

```
git push -u origin <branch>
```

Safer force push

```
git push --force-with-lease
```

TRAP origin/main does not update by itself. It is a snapshot from your last fetch. If git status says "up to date", it means up to date with that snapshot, not with the server right now.

TRAP The lease compares with origin/branch. If an editor or a script runs git fetch in the background, origin/branch is already updated and the lease passes without you ever seeing Ben's work. Git 2.30 added --force-if-includes, used together with --force-with-lease, to also check that you had integrated what you fetched.

4.2 Team workflows, pull requests and branch protection

Workflows GitHub flow: main plus short branches. Gitflow: main plus develop, release and hotfix branches. Trunk based: merge small changes daily behind feature flags. Forking: push to your fork, PR upstream.

PR and MR Server pages that show the diff, run CI and collect reviews before a merge. Keep them small.

Merge methods Merge commit keeps the branch, squash gives one commit, rebase or fast forward gives a straight line.

Protect main Require PRs, approvals and passing status checks (GitHub) or Pipelines must succeed (GitLab). Block force pushes and deletes.

CODEOWNERS and messages CODEOWNERS: last match wins, owners auto requested. Conventional Commits: fix PATCH, feat MINOR, ! or BREAKING CHANGE MAJOR.

EXAM Gitflow was published by Vincent Driessen in 2010. In 2020 he added a note to the article saying that for software delivered continuously, such as web apps, a simpler workflow like GitHub flow is usually a better fit. Gitflow still suits software with explicit versions.

TRAP Squash and merge makes main easy to read, but the branch commits do not appear on main. If someone keeps working on the old branch afterwards, Git does not know those commits were merged and conflicts follow. Start a fresh branch after a squash merge.

TRAP If your branch rule requires a status check called unit-tests, the job name in the workflow must stay unit-tests. Renaming the job makes the required check wait forever. GitHub also advises keeping job names unique across workflows for this reason.

4.3 Power tools: bisect, worktree, submodules, hooks and signing

git bisect start, bad, good , then good or bad or skip until the first bad commit, then reset. bisect run script: 0 good, 125 skip, 1 to 127 bad. About log2(n) tests.

worktree git worktree add gives another folder on another branch, sharing one .git. One branch per worktree.

Submodules Pinned commit of another repository. .gitmodules holds the URL. Clone with --recurse-submodules or run submodule update --init --recursive. CI needs submodules: recursive or GIT_SUBMODULE_STRATEGY.

Hooks pre-commit, commit-msg, pre-push on the client (skippable with --no-verify, not cloned), pre-receive and post-receive on the server. Share with core.hooksPath or the pre-commit framework, and repeat checks in CI.

More tools git grep searches tracked files, sparse-checkout limits folders, commit -S and tag -s sign (SSH or GPG), git archive exports a tag without .git.

Add and clone

```
git submodule add <url> <path>, git clone --recurse-submodules <url>
```

After a pull

```
git submodule update --init --recursive
```

Search tracked files

```
git grep -n "def add"
```

Sign a commit or tag

```
git commit -S, git tag -s v1.0.0
```

TRAP Files in .git/hooks are not committed and not cloned, and anyone can skip them with --no-verify. Use hooks for fast feedback on your laptop, and run the same checks again in CI, which no one can skip.

STEP 5 Git internals

5.1 Inside .git: blobs, trees, commits and tags

Four objects blob (content), tree (names, modes, ids), commit (tree, parents, author, committer, message), annotated tag (object, type, name, tagger, message).

Object id SHA-1 of the type, a space, the size, a zero byte and the content. 40 hex digits (SHA-256: 64). Stored zlib compressed at .git/objects/xx/rest.

Plumbing to inspect git hash-object [-w], git cat-file -t, -s, -p, git rev-parse. Tree modes: 100644 file, 100755 executable, 120000 symlink, 040000 tree, 160000 submodule.

Merkle chain Commit ids depend on every file and every parent, so changing history changes every later id. Identical content is stored once.

SHA-1 and SHA-256 SHAttered 2017, SHA-1DC since Git 2.13, --object-format=sha256 since 2.29, planned default in Git 3.0.

Hash without storing
`git hash-object <file>`

Read any object
`git cat-file -t <id>, -s, -p`

EXAM The id depends only on the content, never on the file name, date or computer. Ten copies of the same file in a repository are stored once.

TRAP SHA-1 and SHA-256 repositories cannot simply push to each other yet, and some hosting services do not accept SHA-256 repositories. Check your server before you switch.

5.2 Refs, the index, plumbing and packfiles

Refs Names for ids: refs/heads (branches), refs/tags, refs/remotes, packed-refs. HEAD is a symbolic ref to a branch. Change refs with update-ref, never by hand. reftable comes in Git 2.45 and is planned as default in 3.0.

Plumbing behind add and commit hash-object -w, update-index --add, write-tree, commit-tree, update-ref. Porcelain is for people, plumbing is stable for scripts.

Index stages 0 normal, 1 base, 2 ours, 3 theirs during a conflict. git ls-files --stage shows them.

Packs and gc Loose objects are packed by gc, similar objects stored as deltas. gc prunes unreachable objects after two weeks, reflog entries keep commits alive for 30 or 90 days. fsck checks and lists dangling objects.

CI clones actions/checkout fetches one commit by default (fetch-depth: 1). Use fetch-depth: 0 when a job needs tags or history. GitLab uses GIT_DEPTH.

Pack and clean `git gc`

Check the database `git fsck`

TRAP Never edit ref files with a text editor. Use git update-ref, which writes safely, checks the old value if you ask and records the change in the reflog.

EXAM Plumbing output is stable and meant for scripts. Porcelain output such as git status can change between versions, so scripts should use plumbing or the --porcelain options.

TRAP Commits you can still see in the reflog are reachable from it, so gc keeps them until the reflog entry expires (90 days by default, 30 for entries that are not reachable from the branch). That is the safety net from chapter 3.1. --prune=now on a repository you care about removes that net for loose objects nothing points to.

STEP 6 CI/CD foundations

6.1 CI/CD foundations: pipelines, runners, artifacts and roles

CI and CD CI: merge small changes often, every push built and tested. Continuous delivery: always releasable, a person releases. Continuous deployment: a green pipeline releases by itself.

Pipeline parts Pipeline or workflow run, jobs on fresh runners, steps or script lines. Stages wait for the whole previous stage, needs wait only for named jobs. Time = longest path.

YAML Spaces not tabs, key: value, - list items, run: | for blocks. Quote versions ("3.10").

Artifacts versus cache Artifacts are results to keep and pass on (build packages, reports). Cache is a speed up for dependencies, safe to lose.

Secure and promote Secrets outside YAML, least permissions, pin third party actions by SHA (tj-actions 2025, Codecov 2021). Build once, promote the same artifact through dev, QA, staging and production (Knight Capital 2012).

Secrets
GitHub: `${{ secrets.NAME }}`. GitLab: masked and protected CI/CD variables

Least privilege
GitHub: `permissions: contents: read`. GitLab: protected variables only on protected branches

Pin third party actions
`uses: owner/action@<full commit SHA>`

TRAP YAML guesses types. python-version: 3.10 is read as the number 3.1, and the job installs the wrong Python. Always quote versions: python-version: "3.10". Old YAML 1.1 parsers also read a bare no or off as false.

EXAM If each environment rebuilds from source, QA tested a different file from the one users get. Build once, store it as an artifact or in a package registry, and promote that same file.

STEP 7 GitHub Actions

7.1 GitHub Actions: events, jobs, matrix and contexts

Workflow file .github/workflows/*.yml with on, jobs, runs-on, steps (uses or run). checkout first. Quote versions.

Events push and pull_request with branches, tags, paths filters, workflow_dispatch with inputs, schedule cron in UTC on the default branch. * stops at /, ** does not.

Jobs Parallel by default, needs for order, if for conditions, matrix for combinations (max 256, fail-fast default true). Outputs through $GITHUB_OUTPUT.

Contexts github, env, vars, secrets, matrix, steps, needs, inputs, runner inside ${{ }}.

Safety permissions least privilege, concurrency with cancel-in-progress, environments with reviewers, untrusted text through env only, fork PRs get no secrets, job timeout default 360 minutes.

Branch and tag filters
`branches: [main, 'release/**'], tags: ['v*']`

Cron `- cron: '30 2 * * 1-5'`

Least privilege token
`permissions: contents: read`

One run per branch
`concurrency: group: ci-${{ github.ref }} cancel-in-progress: true`

TRAP A workflow added on a feature branch with on: push: branches: [main] does not run for pushes to that feature branch. Beginners often think CI is broken. Add pull_request, open a PR, or push to main.

EXAM A matrix can produce at most 256 jobs per workflow run. Six operating systems times six Python versions times eight databases is already 288.

TRAP Pull requests from forks get a read only GITHUB_TOKEN and no repository secrets. Do not switch to pull_request_target to work around this if the job runs code from the pull request, because that event runs with your secrets.

7.2 GitHub Actions from scratch: a green build on every commit

From scratch Test locally, create an empty repo, commit the project with .github/workflows/ci.yml, push to main, watch the Actions tab, download the artifact, break it on a branch, fix it, then require the checks on main.

ci.yml push and pull_request on main plus workflow_dispatch, permissions contents: read, concurrency per ref. lint, unit-tests, build, smoke-tests chained by needs. Checkout in every job.

Artifacts in the flow build uploads calc-app once (if-no-files-found: error), smoke-tests downloads it with download-artifact in the same run, reports kept with if: always().

Dev versus QA changes Dev: triggers, matrix, cache, cancel-in-progress. QA: test the artifact, separate scheduled suites, fail-fast false, longer retention, inputs.

QA regression schedule plus workflow_dispatch input, actions: read, GH_TOKEN, gh run list for the last green run, gh run download -n calc-app, inputs passed through env.

7.3 GitHub Actions artifacts and caching: publish, pull and speed up

upload-artifact name (unique per run), path with ! excludes, if-no-files-found, retention-days (default 90), outputs artifact-id, artifact-url, artifact-digest. Immutable since v4, overwrite: true replaces. Hidden files skipped, permissions reset.

Pull artifacts Same run: download-artifact name or pattern plus needs. Other run: github-token, repository, run-id. People: run page. Terminal: gh run download -n name -D dir. API: /repos/{owner}/{repo}/actions/artifacts/{id}/zip.

actions/cache key from runner.os and hashFiles, restore-keys prefixes, cache-hit only on exact match, saved only on a miss. Branch scope plus default branch. 10 GB per repo, 7 days unused.

Choose Artifact for results of a run, cache for speed, Release or Packages for versions users install.

Safety No secrets in caches or artifacts, read only cache for fork triggers, record digests, use attestations for release builds.

TRAP A matrix job that uploads with a fixed name fails from the second combination on, because names must be unique. Add the matrix values: name: report-${{ matrix.os }}-${{ matrix.python }}.

EXAM Artifacts are for files of one run, kept for a limited time. For a release that users install, publish it to GitHub Releases or GitHub Packages, which keep it until you delete it.

TRAP Never cache secrets or build outputs that must be correct. Caches can be restored from a partial match, so they may hold stale files. If a job must use exactly these files, it needs an artifact, not a cache.

STEP 8 GitLab CI/CD

8.1 GitLab CI/CD: stages, rules, needs and runners

File and stages .gitlab-ci.yml at the root. Default stages .pre, build, test, deploy, .post. Same stage in parallel, next stage after the previous one succeeds. script is required.

Rules Top down, first match decides, when never excludes, no match excludes. workflow:rules decide the whole pipeline and stop duplicate branch plus MR pipelines.

Pipeline types Branch (CI_COMMIT_BRANCH), tag (CI_COMMIT_TAG), merge request (CI_PIPELINE_SOURCE merge_request_event), schedule, web.

Speed and reuse needs (up to 50, needs: [] starts at once), parallel N or matrix (up to 200), hidden .jobs with extends, include for shared YAML.

Runners and variables Instance, group, project runners. Docker, shell, Kubernetes executors. tags select runners. Masked and protected variables for secrets.

Skip waiting for the stage
`needs: [build]`

Run a job many times
`parallel: 3` or `parallel: matrix:`

Project variables Settings, CI/CD, Variables

EXAM script lines run in order, and the job fails at the first command that exits with a non zero code. after_script still runs, but its result does not change the job status.

TRAP Add a rule for merge requests to one job without workflow:rules and a single push to a branch with an open merge request can start two pipelines, a branch pipeline and a merge request pipeline. The workflow block above is the documented fix.

8.2 GitLab CI/CD from scratch: a green pipeline on every commit

From scratch Blank project without README, push the repo with .gitlab-ci.yml, watch Build, Pipelines, check the Tests tab and job artifacts, break and fix in a merge request, then Pipelines must succeed and protected main.

The sample file workflow rules (MR, skip branch with open MR, branches), stages lint, test, build, qa, image python:3.13-slim, cache key files on requirements.txt, junit reports, build artifact with name and expire_in, smoke-tests with needs artifacts, nightly job on schedule.

Tools Pipeline editor (Validate, Visualize), Tests tab, job artifacts Download and Browse, Build, Pipeline schedules.

Dev versus QA changes Dev: needs, interruptible, cache, extends, protected variables. QA: test the artifact, parallel suites, JUnit reports, schedules and web runs, longer expiry for releases.

Common errors No pipeline (YAML or workflow rules), stuck job (runner tags), duplicate pipelines (no workflow rules), missing artifact (needs), cache path outside the project.

EXAM GitLab uploads artifacts of all jobs and passes them on to jobs in later stages automatically. needs and dependencies control which ones a job downloads. dependencies: [] downloads none, which saves time in jobs that do not need files.

8.3 GitLab artifacts and caching: keep, pass on and pull builds

artifacts keyword paths, exclude, name, when (on_success, on_failure, always), expire_in (GitLab.com default 30 days), access, reports: junit. Latest successful artifacts per ref are kept beyond expire_in. Keep button. 1 GB on GitLab.com.

Between jobs Earlier stages by default, dependencies: [list] or [] to narrow, needs: [job] to start early and fetch, needs:project for other projects (Premium).

Pull artifacts UI: pipelines, jobs, job page, MR, Build Artifacts. API: /projects/:id/jobs/artifacts/:ref/download?job=name, /jobs/:job_id/artifacts?file_type=junit. CLI: glab job artifact . CI_JOB_TOKEN with JOB-TOKEN header plus allowlist.

Cache cache key files (up to two), prefix, paths inside the project, policy pull-push, pull or push. Best effort only.

Releases Artifacts expire. Releases go to the package registry or release assets.

TRAP reports such as junit are not inside the downloadable archive. To keep the XML as a file too, also list it under paths, as the sample project does.

EXAM The finished release belongs in the package registry or a release asset, not in a job artifact that may expire. For example, publish a Python wheel to the project's PyPI registry in a tag pipeline with the CI_JOB_TOKEN.

STEP 9 Exam revision

9.1 Exam revision: commands, scenarios and pipeline bug hunts

EXAM When asked "how do you undo", first say where the mistake is (working tree, staging area, local commit, pushed commit) and whether others have it. The right command follows from those two facts.